

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****ASSESSMENT TOOL 1 – Prompt Engineering Challenge**

Course Code:	CSA65	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL1, BL2)	Assessment:	Assessment Tool 3 – Prompt Engineering Challenge
Weightage:	40% of CIA	Total Marks:	100 (2 Question)
CO1 Statement:	<i>Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems.(BL1, BL2)</i>		
Student Name:	Ch Rama karthik Reddy	Reg. No.:	192472281
Section / Batch:	2024	Date:	29/7/2026

Instructions to Students

- Answer 2 questions. Each question carries 50 Marks. Total: 100 Marks.
- Write clear, well-structured prompts and explanations where asked.
- Use your own original examples — do not copy from classmates or the internet verbatim.
- Where a diagram or worked example is requested, label it clearly.
- Illustrate prompting techniques (zero-shot, few-shot, chain-of-thought, role/system prompting, etc.) with concrete examples, not just definitions. Time allotted: 30 mins

Code of Conduct

I karthik/192472281 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – PROMPT ENGINEERING CHALLENGE**Q14. Retrieval-Augmented Prompting**

Write a prompt template that injects retrieved context chunks (e.g., snippets from a company FAQ) into the prompt, and instructs the model to answer strictly from that context — explicitly refusing to answer if the context doesn't contain the information.

Q15. Analogical Prompting

Write a prompt that asks the model to first recall 1–2 similar problems it has solved before, then use that reasoning to solve a new problem — for example, debugging a new piece of code by referencing a similar bug pattern.

Q14. Retrieval-Augmented Prompting

Write a prompt template that injects retrieved context chunks (e.g., snippets from a company FAQ) into the prompt, and instructs the model to answer strictly from that context — explicitly refusing to answer if the context doesn't contain the information.

Prompt

You are a helpful AI assistant.

Use ONLY the information provided in the retrieved context below to answer the user's question.

Retrieved Context:

{retrieved_context}

Instructions:

1. Read the retrieved context carefully.
2. Answer ONLY using the retrieved context.
3. Do NOT use external knowledge or make assumptions.
4. If the answer is not present in the retrieved context, reply exactly:

"I don't have enough information in the provided context to answer this question."

5. Keep your answer concise and accurate.

User Question:

{user_question}

Input:

Can I return a product after 45 days?

Output:

No. According to the provided context, products can be returned within 30 days of purchase if they are unused and in their original packaging.

Python Code:

```
from groq import Groq

# Replace with your Groq API key

client =

Groq(api_key=" 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")

# Retrieved context (Example: Company FAQ)

retrieved_context = """

Company FAQ

Q: What is the return policy?

A: Products can be returned within 30 days of purchase if they are unused and in their original packaging.

Q: What are the customer support hours?

A: Customer support is available Monday to Friday from 9:00 AM to 6:00 PM IST.

Q: Do you offer international shipping?

A: Yes, we ship to over 50 countries worldwide.

"""

# User question

user_question = "Can I return a product after 45 days?"
```

Prompt Template

```
prompt = f"""
```

You are a helpful AI assistant.

Use ONLY the information provided in the retrieved context below to answer the user's question.

Retrieved Context:

```
{retrieved_context}
```

Instructions:

1. Read the retrieved context carefully.
2. Answer ONLY using the retrieved context.
3. Do NOT use external knowledge or make assumptions.
4. If the answer is not present in the retrieved context, reply exactly:

"I don't have enough information in the provided context to answer this question."

5. Keep your answer concise and accurate.

User Question:

```
{user_question}
```

```
"""
```

Send request to Groq

```
response = client.chat.completions.create(
```

```
    model="llama-3.3-70b-versatile",
```

```
    messages=[
```

```
{"role": "user", "content": prompt}
```

```
],
```

```
temperature=0
```

```
)
```

```
# Print the output
```

```
print(response.choices[0].message.content)
```

Output:

```
145.11/145.11 KB 0.4 MB/s eta 0.00.00
```

```
Installing collected packages: groq
```

```
Successfully installed groq-1.6.0
```

```
No, according to the return policy, products can be returned within 30 days of purchase if they are unused and in their original packaging.
```

Q15. Analogical Prompting

Write a prompt that asks the model to first recall 1–2 similar problems it has solved before, then use that reasoning to solve a new problem — for example, debugging a new piece of code by referencing a similar bug pattern.

Prompt

You are an expert software engineer.

Before solving the new problem, follow these steps:

1. Recall 1–2 similar debugging problems you have solved before.
2. Briefly explain the bug pattern in those problems.
3. Compare those bug patterns with the new problem.
4. Apply the same reasoning to solve the new problem.
5. Explain your solution step by step.
6. Provide the corrected code.
7. Explain why your solution works.

New Problem:

Debug the following Python code:

```
def find_average(numbers):  
  
    total = 0  
  
    for i in range(len(numbers)+1):  
  
        total += numbers[i]  
  
    return total / len(numbers)
```

```
print(find_average([10,20,30,40]))
```

Python code:

```
from groq import Groq
```

```
# Replace with your Groq API Key
```

```
client = Groq(api_key="YOUR_GROQ_API_KEY")
```

```
prompt = """
```

You are an expert software engineer.

Before solving the new problem, follow these steps:

1. Recall 1–2 similar debugging problems you have solved before.
2. Briefly explain the bug pattern in those problems.
3. Compare those bug patterns with the new problem.
4. Apply the same reasoning to solve the new problem.
5. Explain your solution step by step.
6. Provide the corrected code.
7. Explain why your solution works.

New Problem:

Debug the following Python code:

```
def find_average(numbers):
```

```
    total = 0
```

```
    for i in range(len(numbers)+1):
```

```
        total += numbers[i]

    return total / len(numbers)

print(find_average([10,20,30,40]))

response = client.chat.completions.create(

    model="llama-3.3-70b-versatile",

    messages=[

        {

            "role": "user",

            "content": prompt

        }

    ],

    temperature=0.3

)

print(response.choices[0].message.content)
```

Output:

```
def find_average(numbers):
    total = 0
    for i in range(len(numbers)):
        total += numbers[i]
    return total / len(numbers)

print(find_average([10, 20, 30, 40]))
```

25.0

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technique Selection	30%	Correctly applies zeroshot/oneshot/few-shot & CoT technique appropriate to each task	Applies techniques with minor mismatch to task	Limited understanding of which technique to use	Techniques applied incorrectly or not evident
Output Effectiveness	25%	Prompts produce accurate, taskappropriate outputs across all assigned tasks	Prompts work well for most tasks	Prompts work for only a few tasks	Prompts are largely ineffective
Prompt Craftsmanship	20%	Prompts are clear, specific, and reusable	Prompts are mostly clear	Prompts are vague or inconsistent	Prompts are poorly constructed
Prompt-Output Documentation	15%	Welldocumented prompt-output pairs with clear rationale	Documented with minor gaps	Minimal documentation of prompts/outputs	No rationale provided for prompt choices
Design Rationale	10%	Clearly explains design choices and iterations made	Explains most design choices	Limited explanation of process	No explanation of process provided

Marks Evaluation:

Question No.	Technique Selection (30%)	Output Effectiveness(25%)	Prompt Craftsmanship (20%)	Prompt-Output Documentation (15%)	Design Rationale(10%)	100
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____